

CUDA: Fundamentos, Modelo de Hilos, Modelo de Memorias y Optimización

Brian David Acevedo Gomez – Código: 1092526700

Geron Jose Vergara Garcia – Código: 1116776436

Fundamentos de Computación Paralela y Distribuida

Cúcuta, Norte de Santander, Colombia

Mayo 2026

I. FUNDAMENTACIÓN CUDA

CUDA (Compute Unified Device Architecture) es una plataforma de computación paralela y un modelo de programación desarrollado por NVIDIA en 2007, que permite a los desarrolladores utilizar la unidad de procesamiento gráfico (GPU) para propósitos de cómputo general, más allá del renderizado de gráficos. Esta tecnología surgió como respuesta a la creciente demanda de potencia computacional en áreas como inteligencia artificial, simulaciones científicas, procesamiento de señales y minería de datos.

La arquitectura CUDA se fundamenta en el paradigma de programación SIMT (Single Instruction, Multiple Threads), en el cual una misma instrucción es ejecutada simultáneamente por miles de hilos en los núcleos de la GPU. A diferencia de la CPU, que cuenta con pocos núcleos altamente optimizados para tareas secuenciales, la GPU está compuesta por cientos o miles de núcleos más sencillos diseñados para el procesamiento masivamente paralelo. NVIDIA introdujo este modelo con la familia de GPUs G80, habilitando el acceso a los recursos de hardware mediante un conjunto de instrucciones y una API basada en lenguaje C/C++ extendido.

Desde el punto de vista de la arquitectura de hardware, CUDA organiza los recursos computacionales en unidades llamadas Streaming Multiprocessors (SMs). Cada SM contiene un conjunto de núcleos CUDA (CUDA cores), unidades de función especial, registros, caché L1 y memoria compartida. La interacción entre el programa del lado del host (CPU) y el programa del dispositivo (GPU) se realiza mediante el modelo de ejecución de kernels, que son funciones definidas en el código CUDA y ejecutadas en la GPU en forma masivamente paralela.

Uno de los aspectos más relevantes de la fundamentación de CUDA es su abstracción de la jerarquía de cómputo: el programador define la cantidad de hilos, su organización en bloques y la disposición de estos en una grilla (grid), permitiendo escalar el paralelismo de acuerdo a las características del hardware disponible. Esta abstracción garantiza portabilidad y escalabilidad entre diferentes generaciones de GPUs NVIDIA, haciendo de CUDA una plataforma robusta para el desarrollo de aplicaciones científicas y de alto rendimiento.

Referencias:

- [1] NVIDIA Corporation, "CUDA C++ Programming Guide," NVIDIA Documentation, 2023. [En línea].
Disponible en: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>
- [2] J. Sanders y E. Kandrot, CUDA by Example: An Introduction to General-Purpose GPU Programming. Addison-Wesley Professional, 2010.
- [3] Wikipedia, "CUDA," 2024. [En línea]. Disponible en: <https://es.wikipedia.org/wiki/CUDA>

II. MODELO DE HILOS EN CUDA

El modelo de hilos de CUDA define una jerarquía de tres niveles para organizar la ejecución paralela en la GPU: hilos (threads), bloques (blocks) y grillas (grids). Un hilo es la unidad mínima de ejecución; cada hilo ejecuta el mismo kernel pero puede operar sobre datos distintos, identificándose mediante variables de posición como `threadIdx`, `blockIdx` y `blockDim`. Esta organización permite que el programador diseñe algoritmos altamente paralelos donde miles de hilos trabajan simultáneamente.

Los hilos se agrupan en bloques, y todos los hilos de un mismo bloque se ejecutan en el mismo Streaming Multiprocessor (SM), lo que les permite compartir memoria de alta velocidad (memoria compartida) y sincronizarse mediante la función `__syncthreads()`. Un bloque puede contener hasta 1024 hilos, y estos pueden organizarse en dimensiones de una, dos o tres dimensiones, facilitando el mapeo natural sobre datos matriciales o volumétricos. Múltiples bloques se agrupan en una grilla (grid), que también puede ser unidimensional, bidimensional o tridimensional.

A nivel de hardware, los hilos se ejecutan en grupos de 32 denominados warps. Un warp es la unidad básica de planificación del SM: todos los hilos de un warp ejecutan la misma instrucción en el mismo ciclo de reloj. Si los hilos de un warp siguen caminos de ejecución distintos (divergencia de warp), el hardware serializa las ramas, reduciendo la eficiencia. Por ello, minimizar la divergencia de control dentro de un warp es una práctica fundamental para maximizar el aprovechamiento del hardware.

El concepto de ocupancia (occupancy) describe qué tan eficientemente se utilizan los recursos del SM: hace referencia al cociente entre el número de warps activos y el máximo de warps que el SM puede alojar. Una alta ocupancia no garantiza necesariamente el mejor rendimiento, ya que depende también del uso de registros y memoria compartida por hilo. La correcta configuración de la dimensión de los bloques, eligiendo múltiplos de 32 (el tamaño del warp), es esencial para maximizar la eficiencia en el modelo de hilos de CUDA.

Referencias:

- [1] NVIDIA Corporation, "CUDA C++ Best Practices Guide," NVIDIA Documentation, 2023. [En línea]. Disponible en: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- [2] D. Kirk y W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, 3rd ed. Morgan Kaufmann, 2016.
- [3] Reddit Community, "How does CUDA blocks/warps/thread works?," r/CUDA, 2022. [En línea]. Disponible en: <https://www.reddit.com/r/CUDA/comments/x2f767/>

III. MODELO DE MEMORIAS EN CUDA

El modelo de memorias de CUDA establece una jerarquía de distintos tipos de memoria, cada uno con diferente velocidad de acceso, capacidad y alcance. Esta jerarquía es fundamental para el diseño de kernels eficientes, ya que el uso inadecuado de los niveles de memoria puede degradar significativamente el rendimiento. El modelo asume que el host (CPU) y el dispositivo (GPU) mantienen espacios de memoria separados, y que los datos deben transferirse explícitamente entre ambos antes y después de la ejecución del kernel.

La memoria global es el tipo más amplio y accesible por todos los hilos de cualquier bloque y grilla. Aunque ofrece gran capacidad (varios gigabytes), su latencia de acceso es elevada (400-800 ciclos). Para mitigar esto, CUDA dispone de la memoria compartida (shared memory), que es una memoria on-chip de alta velocidad accesible por todos los hilos de un mismo bloque, con latencias similares a las de los registros y un rendimiento hasta 15 veces superior al de la memoria global. Su tamaño es limitado (típicamente 48 KB por SM en arquitecturas modernas), por lo que debe utilizarse estratégicamente.

Los registros representan el nivel más rápido de memoria en CUDA; son privados por hilo y su acceso no genera latencia adicional. Cada SM dispone de un banco de registros de 32 bits compartido entre todos los hilos activos en ese SM, por lo que el número de registros utilizados por hilo afecta directamente la ocupancia. Adicionalmente, existen la memoria constante, destinada a datos de solo lectura con caché optimizada para accesos en broadcast (cuando todos los hilos de un warp acceden al mismo dato), y la memoria de textura, que ofrece caché especializada para accesos con localidad espacial en patrones bidimensionales.

Una práctica clave en el manejo del modelo de memorias es el acceso coalescente a la memoria global: cuando los hilos consecutivos de un warp acceden a posiciones de memoria contiguas y alineadas, el hardware puede consolidar las transacciones en un solo acceso, reduciendo el número de operaciones de memoria necesarias. Por el contrario, los accesos no coalescentes generan múltiples transacciones independientes, degradando el rendimiento. Optimizar la coalescencia y hacer uso eficiente de la memoria compartida son las dos estrategias de memoria más impactantes para lograr alto rendimiento en aplicaciones CUDA.

Referencias:

- [1] Cocinando Tarjetas Gráficas, "Modelo de Memoria en CUDA," 2013. [En línea]. Disponible en: <http://cocinando-tarjetas-graficas.blogspot.com/2013/09/modelo-de-memoria-en-cuda.html>
- [2] A. Chavarría, "Uso efectivo del subsistema de memoria en GPU con CUDA y Numba," LinkedIn, 2025.
- [3] NVIDIA Corporation, "CUDA C++ Programming Guide – Memory Hierarchy," 2023. [En línea]. Disponible en: <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>

IV. OPTIMIZACIÓN Y EFICIENCIA EN CUDA

La optimización en CUDA es un proceso multidimensional que abarca desde la configuración del lanzamiento de kernels hasta el diseño algorítmico y el manejo de la memoria. El primer principio de optimización es maximizar la ocupancia del SM mediante una configuración adecuada del tamaño de los bloques de hilos. Se recomienda que el número de hilos por bloque sea múltiplo de 32 (tamaño del warp) para evitar warps incompletos y desperdiciar recursos de cómputo. Valores típicos son 128, 256 o 512 hilos por bloque, dependiendo del uso de registros y memoria compartida de cada kernel.

El ancho de banda de la memoria es frecuentemente el cuello de botella principal en aplicaciones CUDA. Por ello, una estrategia fundamental de optimización consiste en minimizar las transferencias entre la memoria del host y la del dispositivo, que son costosas en términos de latencia y tiempo de transferencia a través del bus PCIe. Se recomienda mantener los datos en la GPU el mayor tiempo posible, procesando múltiples kernels consecutivos sin regresar los datos al CPU innecesariamente. La técnica de computación asíncrona con streams de CUDA permite solapar la ejecución de kernels con transferencias de datos, mejorando la utilización del hardware.

La minimización de la divergencia de warp es otra técnica crítica: cuando hilos del mismo warp toman ramas condicionales diferentes, el hardware serializa las ejecuciones, reduciendo la eficiencia a la mitad o más. Reestructurar los algoritmos para que los hilos de cada warp sigan siempre el mismo camino de ejecución mejora sustancialmente el rendimiento. Asimismo, el uso adecuado de la memoria compartida como buffer intermedio —cargando bloques de datos desde la memoria global una sola vez y reutilizándolos múltiples veces— es una de las técnicas de tiling o mosaico más empleadas en algoritmos de álgebra lineal y procesamiento de imágenes.

Finalmente, herramientas de perfilado como NVIDIA Nsight y nvprof permiten identificar cuellos de botella específicos en un kernel, midiendo métricas como la ocupancia real, el número de transacciones de memoria y la eficiencia de instrucción. El proceso de optimización en CUDA debe ser iterativo: se parte del perfil base de la aplicación, se identifican las secciones críticas, se aplican mejoras como coalescencia de memoria, uso de memoria compartida o reducción de divergencia, y se mide el impacto. Este ciclo garantiza mejoras sostenidas en el rendimiento sin comprometer la corrección del código.

Referencias:

- [1] NVIDIA Corporation, "CUDA C++ Best Practices Guide – Performance Metrics," 2023. [En línea]. Disponible en: <https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/>
- [2] D. Kirk y W. Hwu, Programming Massively Parallel Processors: A Hands-on Approach, 3rd ed. Morgan Kaufmann, 2016.
- [3] YouTube, "Optimización y consideraciones de rendimiento en CUDA," 2025. [En línea]. Disponible en: <https://www.youtube.com/watch?v=HRSMTc5dj6U>